Python Activity-Based Learning

Ten Advanced Case Studies

Student: PAMULA.CIYON **Roll:** 26001335 **Branch/Section:** CSM-7

This report documents ten selected case studies using the course syllabus topics. Review the stated assumptions against the faculty's exact criteria before submission.

Case Study 1: University Scholarship Allocation

Problem statement

Screen and rank applicants using marks, attendance, and income.

Objectives

Use tuples, multiple eligibility conditions, failed-reason lists, and ranking.

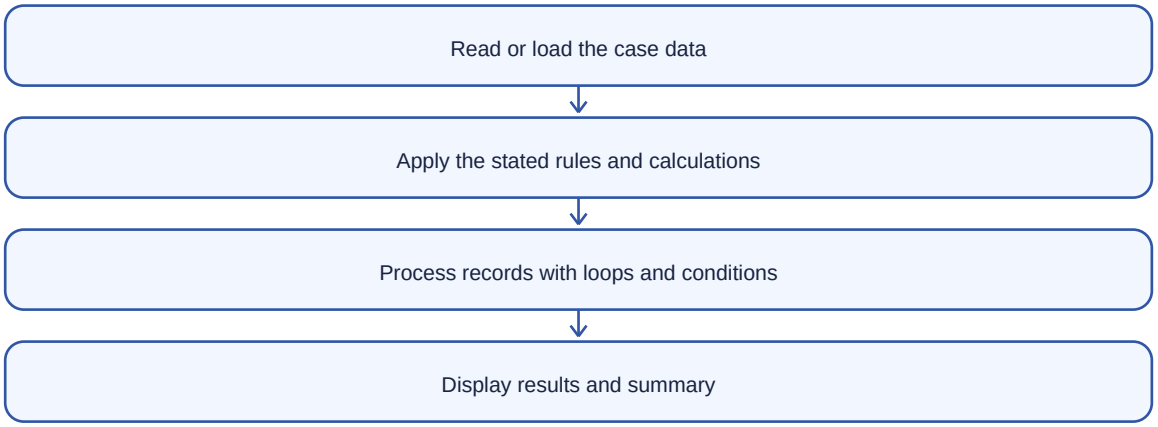
Assumptions

Assume aggregate>=75, subject minimum>=60, attendance>=75, income<=300000; ties use attendance then lower income.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Asha is eligible and ranks first.

Boundary case Charan is rejected for income.

Invalid or exceptional case Applicants below any threshold are rejected with reasons.

Python source code

```
# Case 1: University Scholarship Allocation
records=[("Asha",88,70,92,250000),("Bala",82,65,95,220000),("Charan",76,61,80,310000),("Divya",88,68,92,240000)]
eligible=[]
for name,agg,sub,att,income in records:
    why=[]
    if agg<75: why.append("aggregate")
    if sub<60: why.append("subject minimum")
    if att<75: why.append("attendance")
    if income>300000: why.append("income")
    if len(why)==0: eligible.append((name,agg,att,income))
    else: print(name,"rejected:",", ".join(why))
for i in range(len(eligible)):
```

```
    for j in range(len(eligible)-i-1):
        a=eligible[j]; b=eligible[j+1]
        swap=a[1]<b[1] or (a[1]==b[1] and a[2]<b[2]) or (a[1]==b[1] and a[2]==b[2] and a[3]>b[3])
        if swap: eligible[j],eligible[j+1]=eligible[j+1],eligible[j]
print("Ranking")
rank=1
for row in eligible: print(rank,row[0],row[1],row[2],row[3]); rank+=1
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 27: Advanced Numeric Pattern Generator

Problem statement

Print a centered symmetric numeric diamond for a supplied half-height.

Objectives

Use nested loops and validation.

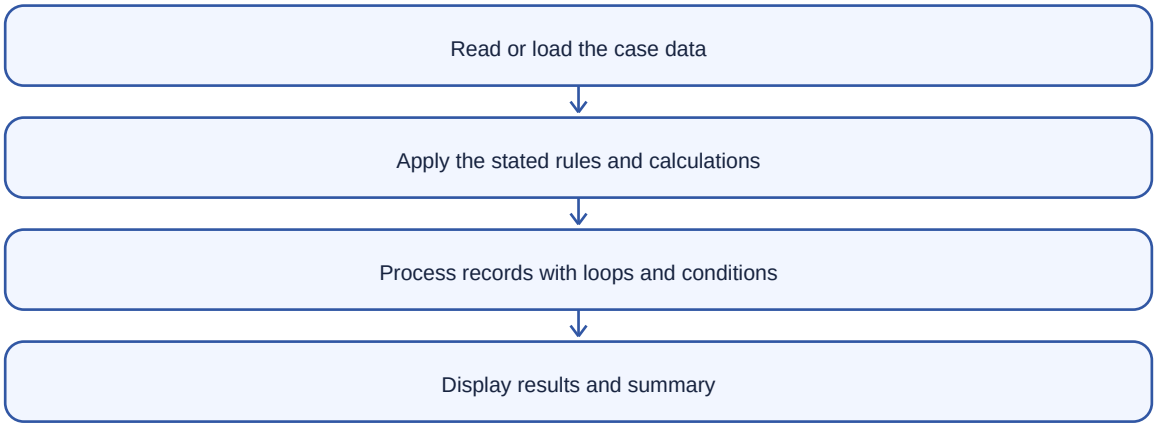
Assumptions

Half-height is 1..6; numbers rise to the row number and fall symmetrically.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Input 3 gives a five-row diamond.

Boundary case Input 1 prints 1.

Invalid or exceptional case Input 7 is rejected.

Python source code

```
# Case 27: Advanced Numeric Pattern Generator
n=int(input("Half-height (1-6): "))
if n<1 or n>6: print("Invalid size")
else:
    for row in range(1,n+1):
        print(" "*(n-row),end="")
        for x in range(1,row+1): print(x,end="")
        for x in range(row-1,0,-1): print(x,end="")
        print()
    for row in range(n-1,0,-1):
        print(" "*(n-row),end="")
        for x in range(1,row+1): print(x,end="")
```

```
        for x in range(row-1,0,-1): print(x,end="")
        print()
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 29: Password Policy Validation Engine

Problem statement

Check a sample password and explain every failed rule.

Objectives

Use strings, loops, membership, and flags.

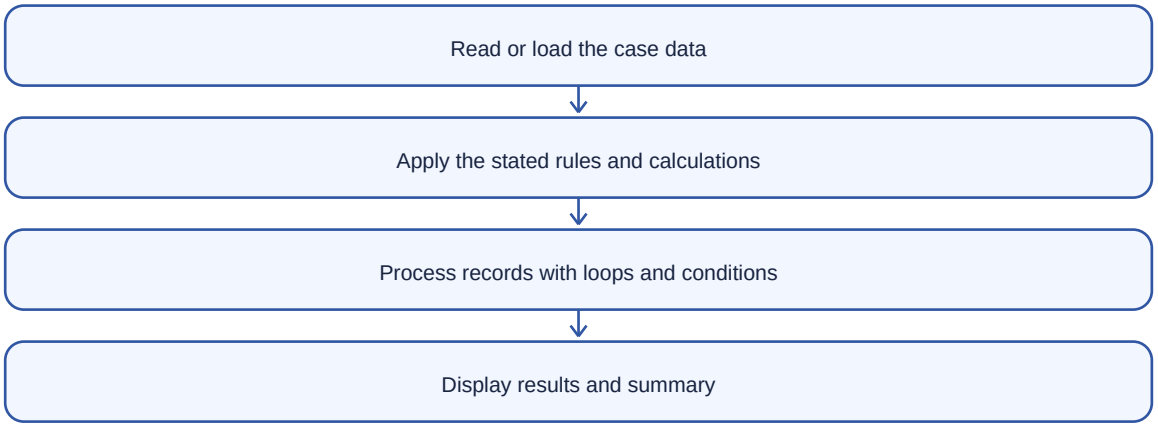
Assumptions

At least 8 chars; uppercase, lowercase, digit, allowed symbol !@#$%&; no spaces or adjacent repeated chars.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Study#2026 passes all rules.

Boundary case short fails several rules.

Invalid or exceptional case Study##2026 fails adjacent-repeat rule.

Python source code

```
# Case 29: Password Policy Validation Engine
p=input("Sample password: ")
u=l=d=s=space=repeat=False
symbols="!@#$%&"
for i in range(len(p)):
    c=p[i]
    if "A"<=c<="Z": u=True
    if "a"<=c<="z": l=True
    if "0"<=c<="9": d=True
    if c in symbols: s=True
    if c==" ": space=True
    if i>0 and c==p[i-1]: repeat=True
```

```
checks=[("length",len(p)>=8),("uppercase",u),("lowercase",l),("digit",d),("symbol",s),("no spaces",not space),("no repeats",not repea
valid=True
for label,ok in checks:
    if ok: print("PASS",label)
    else: print("FAIL",label); valid=False
if valid: print("Valid")
else: print("Invalid")
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 37: Nested Loop Calendar Generator

Problem statement

Print a month from its first weekday and length.

Objectives

Use nested loops for week rows and weekday columns.

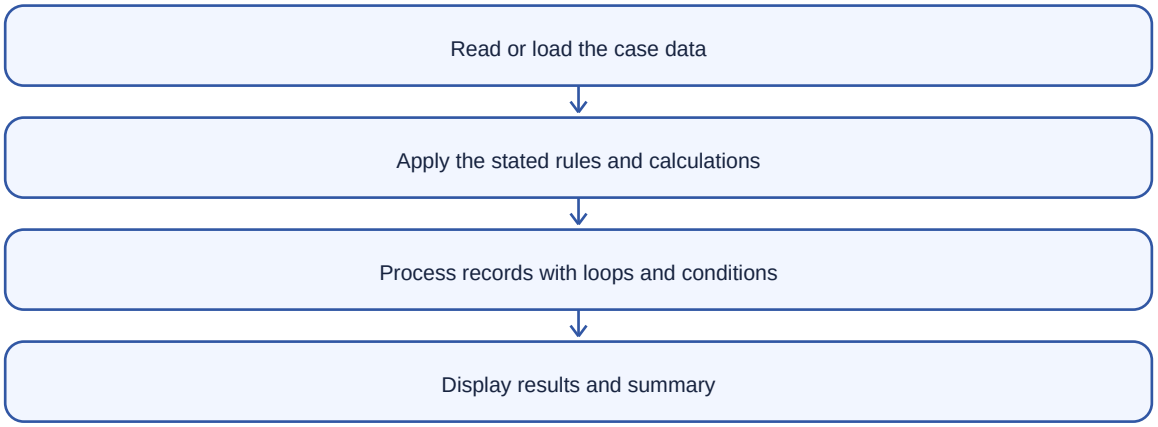
Assumptions

Weekday 0=Sunday..6=Saturday; days 28..31; leap year is out of scope.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample 3,30 places day 1 under Wednesday and ends at day 30.

Boundary case 0,28 starts Sunday.

Invalid or exceptional case Weekday 7 is rejected.

Python source code

```
# Case 37: Nested Loop Calendar Generator
start=int(input("First weekday (0-6): "))
days=int(input("Days (28-31): "))
if start<0 or start>6 or days<28 or days>31: print("Invalid calendar input")
else:
    print("Sun Mon Tue Wed Thu Fri Sat")
    date=1; week=0
    while date<=days:
        for day in range(7):
            if week==0 and day<start: print("    ",end="")
            elif date<=days: print(str(date).rjust(3),end=" "); date+=1
            else: print("    ",end="")
```

```
            print(); week+=1
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 42: Student Marks Matrix Analysis

Problem statement

Calculate student and subject statistics from a nested marks list.

Objectives

Use nested lists and row/column loops.

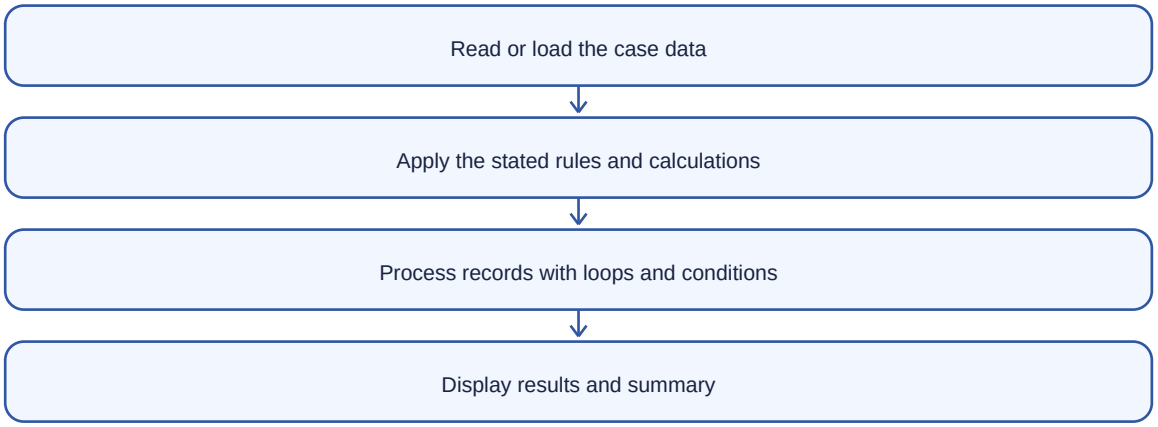
Assumptions

Sample matrix has four students and three subjects; low subject average is below 50.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Divya is top performer.

Boundary case Subject averages are computed by column.

Invalid or exceptional case An average below 50 produces a warning.

Python source code

```
# Case 42: Student Marks Matrix Analysis
students=["Asha","Bala","Charan","Divya"]; subjects=["Python","Maths","Physics"]
marks=[[88,76,91],[64,72,69],[45,54,48],[92,85,89]]
top=0; topavg=-1
for r in range(len(marks)):
    total=0
    for c in range(len(subjects)): total+=marks[r][c]
    avg=total/len(subjects); print(students[r],total,avg)
    if avg>topavg: top=r; topavg=avg
print("Top",students[top],topavg)
for c in range(len(subjects)):
    total=0
```

```
for r in range(len(marks)): total+=marks[r][c]
avg=total/len(marks); print(subjects[c],avg)
if avg<50: print("Low average")
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 47: Sports Tournament Ranking System

Problem statement

Rank teams by points and tie-break rules.

Objectives

Use tuples and nested-loop sorting.

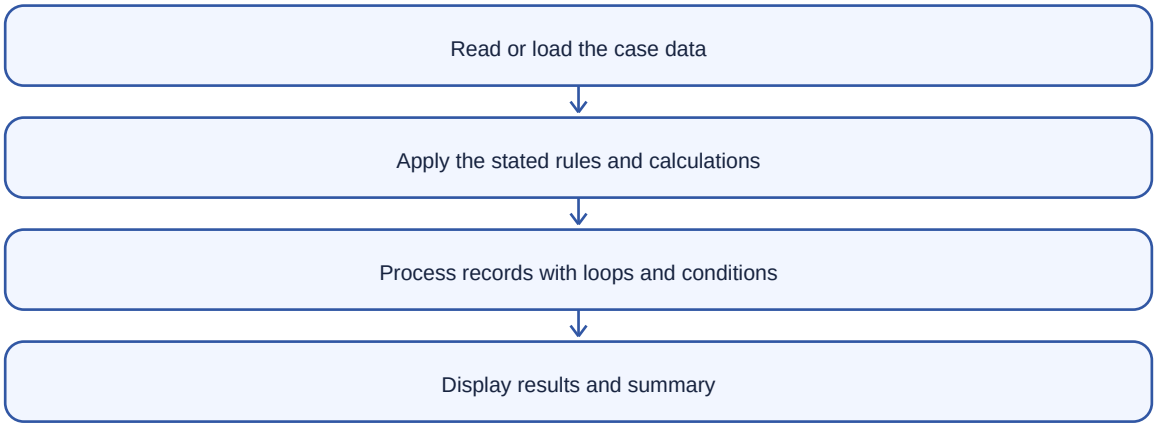
Assumptions

Win=3, draw=1; tie-break by goal difference, goals scored, then name.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Wolves ranks before Falcons on goals scored.

Boundary case Falcons rank before Tigers on goal difference.

Invalid or exceptional case Eagles rank below teams with 7 points.

Python source code

```
# Case 47: Sports Tournament Ranking System
teams=[("Falcons",7,5,9),("Tigers",7,3,8),("Eagles",6,4,7),("Wolves",7,5,10)]
for i in range(len(teams)):
    for j in range(len(teams)-i-1):
        a=teams[j]; b=teams[j+1]
        swap=a[1]<b[1] or (a[1]==b[1] and a[2]<b[2]) or (a[1]==b[1] and a[2]==b[2] and a[3]<b[3]) or (a[1:4]==b[1:4] and a[0]>b[0])
        if swap: teams[j],teams[j+1]=teams[j+1],teams[j]
print("Rank Team Points GD Goals")
rank=1
for name,points,gd,goals in teams:
    print(rank,name,points,gd,goals); rank+=1
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 48: Tuple-Based Employee Shift Conflict Detection

Problem statement

Detect overlapping shifts for the same employee.

Objectives

Compare tuple pairs using interval inequalities.

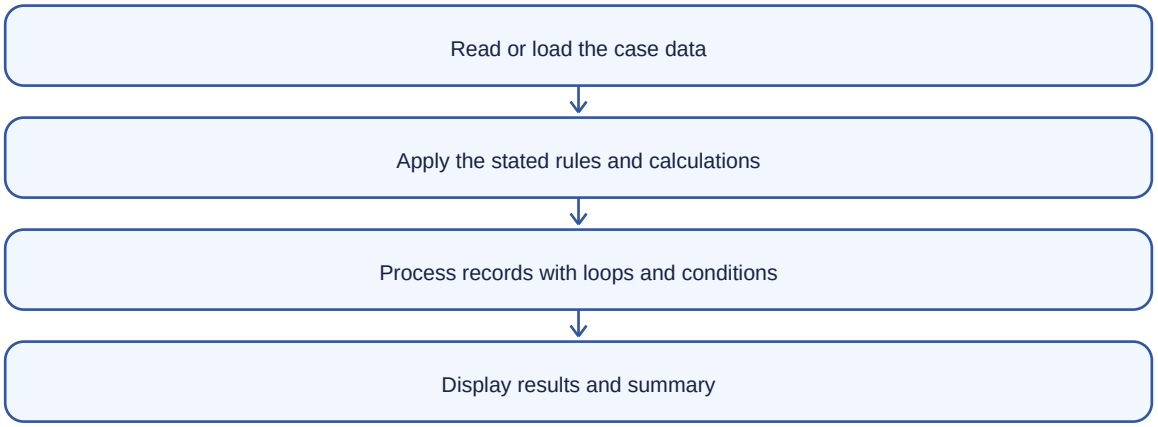
Assumptions

Times are minutes after midnight; endpoint-touching shifts do not overlap.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Asha 540-720 overlaps 690-810.

Boundary case 690-810 and 810-900 only touch, so no conflict.

Invalid or exceptional case Different employees are not compared as conflicts.

Python source code

```
# Case 48: Tuple-Based Employee Shift Conflict Detection
shifts=[("Asha",540,720),("Bala",600,780),("Asha",690,810),("Asha",810,900)]
count=0
for i in range(len(shifts)):
    for j in range(i+1,len(shifts)):
        a=shifts[i]; b=shifts[j]
        if a[0]==b[0] and a[1]<b[2] and b[1]<a[2]:
            print("Conflict",a[0],a[1],a[2],b[1],b[2]); count+=1
if count==0: print("No conflicts")
else: print("Total",count)
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 55: Tuple-Based Transaction Audit

Problem statement

Find repeated IDs, invalid amounts, and suspicious consecutive withdrawals.

Objectives

Use tuple unpacking, nested loops, and counters.

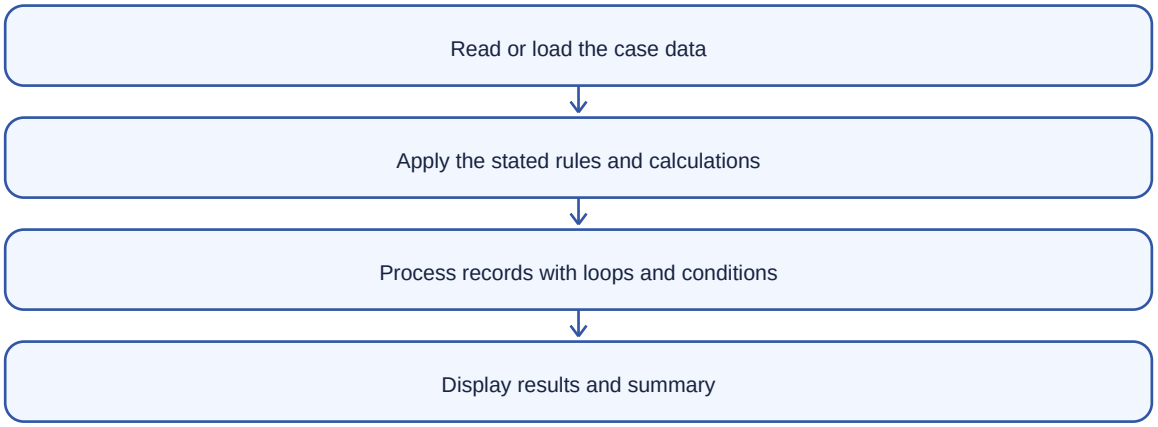
Assumptions

Tuple is ID, amount, type; positive amounts; two consecutive withdrawals over 5000 are flagged.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample T102 duplicate is reported.

Boundary case T104 negative amount is rejected.

Invalid or exceptional case Two consecutive large withdrawals trigger a warning.

Python source code

```
# Case 55: Tuple-Based Transaction Audit
tx=[("T101",1200,"deposit"),("T102",6500,"withdrawal"),("T103",7200,"withdrawal"),("T102",300,"deposit"),("T104",-50,"withdrawal")]
for i in range(len(tx)):
    tid=tx[i][0]; amount=tx[i][1]; kind=tx[i][2]
    if amount<=0: print("Invalid amount",tid,amount)
    if kind!="deposit" and kind!="withdrawal": print("Invalid type",tid)
    for j in range(i+1,len(tx)):
        if tid==tx[j][0]: print("Duplicate ID",tid)
streak=0
for tid,amount,kind in tx:
    if kind=="withdrawal" and amount>5000:
        streak+=1
```

```
            if streak==2: print("Suspicious consecutive large withdrawals")
        else: streak=0
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 66: Student Academic Risk Classification

Problem statement

Combine marks, attendance, and lab scores to classify risk with reasons.

Objectives

Use parallel lists and nested marks lists.

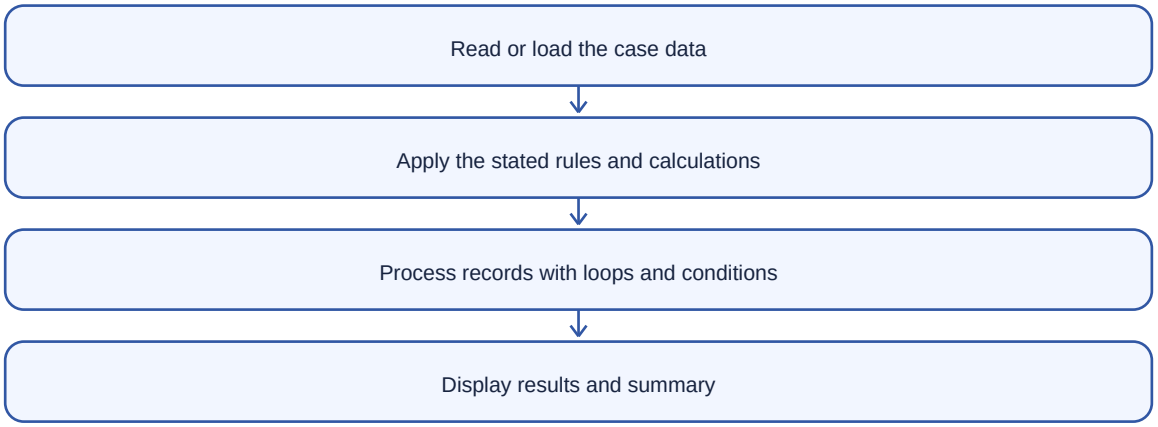
Assumptions

Flags: average <50, attendance <75, lab <40; zero=Low, one=Moderate, two or more=High.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample Asha is Low risk with no flags.

Boundary case Bala is High risk with all three flags.

Invalid or exceptional case Summary counts are displayed.

Python source code

```
# Case 66: Student Academic Risk Classification
names=["Asha","Bala","Charan","Divya"]; marks=[[55,62,58],[42,46,48],[78,81,75],[65,60,70]]
attendance=[82,68,88,74]; lab=[72,35,64,32]; counts=[0,0,0]
for i in range(len(names)):
    total=0
    for m in marks[i]: total+=m
    avg=total/len(marks[i]); reasons=[]
    if avg<50: reasons.append("marks")
    if attendance[i]<75: reasons.append("attendance")
    if lab[i]<40: reasons.append("lab")
    n=len(reasons)
    if n==0: category="Low"; counts[0]+=1
```

```
            elif n==1: category="Moderate"; counts[1]+=1
            else: category="High"; counts[2]+=1
            reason_text=", ".join(reasons)
            if len(reasons)==0: reason_text="no flags"
            print(names[i],avg,category,reason_text)
        print("Low Moderate High",counts)
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.

Case Study 75: Integrated Student Record Consistency Checker

Problem statement

Validate IDs, duplicate records, names, marks, and attendance.

Objectives

Use strings, tuples, loops, and duplicate detection.

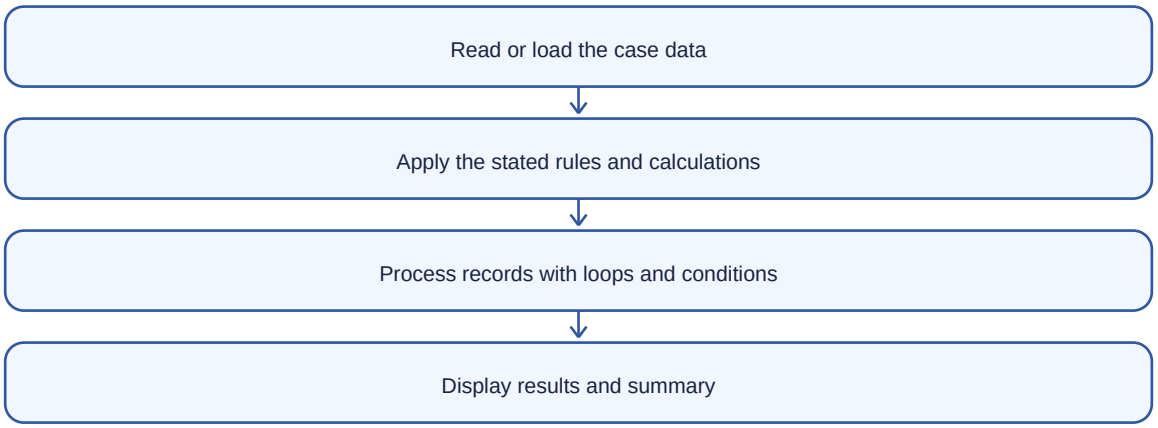
Assumptions

ID is two digits, two uppercase letters, three digits; marks and attendance 0..100.

Algorithm

1. Read or load the case data
2. Apply the stated rules and calculations
3. Process records with loops and conditions
4. Display results and summary

Flowchart



Pseudocode

```
START
  READ or LOAD the required data
  FOR each relevant record
    APPLY the stated conditions and calculations
    STORE or DISPLAY results
  END FOR
  DISPLAY summary
END
```

Test cases

Normal sample 24CS101 duplicate is detected.

Boundary case 24M2102 is malformed and has invalid name/marks.

Invalid or exceptional case 24EC103 has invalid attendance.

Python source code

```
# Case 75: Integrated Student Record Consistency Checker
records=[("24CS101","Asha Rao",86,91),("24CS101","Bala Das",72,78),("24M2102"," ",104,65),("24EC103","Charan Dev",68,-2)]
for i in range(len(records)):
    sid=records[i][0]; name=records[i][1]; marks=records[i][2]; att=records[i][3]
    errors=[]; ok=len(sid)==7
    if ok:
        for p in range(2):
            if not ("0"<=sid[p]<="9"): ok=False
        for p in range(2,4):
```

```
        if not ("A"<=sid[p]<="Z"): ok=False
    for p in range(4,7):
        if not ("0"<=sid[p]<="9"): ok=False
if not ok: errors.append("malformed ID")
for j in range(i+1,len(records)):
    if sid==records[j][0]: errors.append("duplicate ID"); break
if name.strip()=="": errors.append("blank name")
if marks<0 or marks>100: errors.append("invalid marks")
if att<0 or att>100: errors.append("invalid attendance")
if len(errors)==0: print(sid,"VALID")
else: print(sid,", ".join(errors))
```

Sample output screenshot



Conclusion

The program applies the assumptions to its sample data and reports results. Compare the assumptions with the faculty's exact rules before submission.